159.102 · COMPUTER SCIENCE · SECTION 7

Week 7 Exam Notes

Classes, Objects and Pointers — all of Section 7 compressed into glance-cards. Every card = one idea, one memory hook, one exam trap. Read a card in 20 seconds.

01 Classes & objects

02 Timetable program

03 Portable classes

04 Pointer → arrow

05 new: nameless objects

06 Pointers = speed

07 C vs C++ strings

08 String ↔ int

HOW TO USE: 8 cards + 1 recall table. The emoji on each card is the same memory hook from our study session — see the emoji, remember the idea. Read the ⚠ line last: those are the exam traps.

01 Classes & Objects 📄

Class = a blank form. Object = a filled form. One design → many copies. A class keeps data and methods together; methods make main() short and keep the class portable (library-ready).

MEMORY HOOK

Private = locked locker 🔒 (only the class's own methods can touch it). Public = ATM buttons 🔓 (anyone can press). set = put a value in, get = read a value out.

```
class Student {
private:                  // 🔒 locked data
  string name;
public:                   // 🔓 buttons for everyone
  void setname(string n) { name = n; }   // SET = put in
  string getname() { return name; }      // GET = read out
};
Student s;  s.setname("Ali");      // s.name = "Ali"  ✗ ERROR
```

⚠ EXAM WATCH

main() can NEVER write private data directly (s.name = "Ali" fails). set/get methods are the only door. Convention: all data private, all methods public.

02 The Timetable Program

Classes stack like LEGO blocks. session (day, hour, room) sits inside course (code + session), courses sit inside student (name + courses[30]). The program loads a student's courses and reports clashes — two sessions at the same day AND hour.

MEMORY HOOK

Smallest block first: session → course → student. Big block holds smaller blocks.

```
bool session_class::sametime(session_class other) {
  if ((day == other.day) && (hour == other.hour))
    return true;              // same day AND hour = clash
  return false;
}
// BUG: checkclashes() compares every course with EVERY course
// including itself → prints "Clash - 159101 against 159101"
// FIX: skip when i == k  (only compare DIFFERENT courses)
```

⚠ EXAM WATCH

The known bug: a course always clashes with itself — the notes ask "what is the problem with the results?" Answer: self-comparison (i == k), fix by skipping it.

03 Portable Classes

A portable class is a TV remote. Users press the buttons (public methods); nobody cares what's inside. You may change the inside freely — different data types, more variables, new methods — and every program using the class keeps working.

MEMORY HOOK

Change the inside, keep the buttons the same → nobody outside gets hurt.

```
class course_class {
private:
  string code;              // was: int code  (INSIDE changed)
public:
  int getcode() { return stoi(code); }  // still int!
};
// private data changed → public buttons unchanged
// → main() and other programs keep working
```

⚠ EXAM WATCH

Programs using the class do NOT break — as long as the public part stays the same. That's the whole reason data is private.

04 Pointers + the → Arrow →

A pointer just holds an address. Point it at an object with ptr = &obj. Then use the arrow → instead of the dot: it means "go to the object at this address, run the method there."

> MEMORY HOOK
> Object → DOT. Pointer → ARROW. The arrow points at the house the address-note leads to.

```
date_class newyear;          // object  (the house)
date_class *ptr;             // pointer (the address note)
ptr = &newyear;              // note holds the house's address

newyear.setdate(1, 1, 2018);   // object  → DOT
ptr→setdate(1, 1, 2018);       // pointer → ARROW
```

> ⚠ EXAM WATCH
> Mix-up trap: p is a pointer → p→getname() ✓, p.getname() ✗. The object was loaded indirectly through the pointer — newyear was never given the date directly.

05 new — Objects with No Names

new builds a nameless object. No name plate at all — the pointer (holding its address) is the only way to reach it. new reserves the memory, then hands you the address.

> MEMORY HOOK
> Hotel room 🏨: no name on the door — the room number (pointer) is everything.

```
date_class *ptr1, *ptr2;       // pointers only — no objects!
ptr1 = new date_class;       // reserve memory, return address
ptr2 = new date_class;
ptr1→setdate(14, 7, 1789);   // reach it only through the arrow
```

> ⚠ EXAM WATCH
> You MUST use new before ptr→anything — the pointer is just an empty note until new gives it real memory. Without new the program cannot run.

06 Pointers as Parameters — Speed ⚡

Ship the address, not the house. Passing an object copies the WHOLE object into the function (slow for big objects). Passing a pointer copies only the tiny address — much faster. That's why real code often takes pointers-to-objects.

> MEMORY HOOK
> Text your friend the address 🏠 — don't build them a copy of the house.

```
bool sametime(session_class other)  { ... }   // copies WHOLE object
bool sametime(session_class *other) { ... }   // copies address only ⚡
```

> ⚠ EXAM WATCH
> Price of speed: the function gets the SAME object (not a copy) — it can change things inside it.

07 C strings vs C++ strings 🔤

A C string is a char array that ends in \0 — the invisible full-stop that says "string ends here." A C++ string is a class that keeps a C string locked inside and gives you friendly buttons (assign, compare, length()).

> MEMORY HOOK
> \0 = the invisible full-stop. c_str() = the bridge between the two worlds.

```
char str[60];                // C string = char array (max 59 + \0)
"Hi" in memory:  [H][i][\0]   // \0 = stop reading here

// C:  ✗ max length   ✗ str = "Hello"   ✗ if (str1 == str2)
// C++: string s = "Good bye";  ✓ assign  ✓ compare  ✓ s.length()

char *strptr = (char*) str.c_str();        // 🔁 the bridge
datafile.open(filename.c_str(), fstream::out);  // open() wants C string
```

> ⚠ EXAM WATCH
> Old C functions (like open()) need a C string → convert with .c_str(). The counting() example walks letter by letter until it hits \0.

08 String ↔ int Conversion 🔁

stoi turns text into a number; to_string turns a number into text. Text glues ("25"+"25" = "2525"); numbers do real math (25+25 = 50). Convert before doing math.

> MEMORY HOOK
> stoi = **st**ring **to** **i**nt. The notes left this as a TODO comment inside getcode() — stoi fills the gap.

```
int    n = stoi("159101");       // text → number
string s = to_string(159101);      // number → text

"25" + "25"  →  "2525"          // text just glues 😬
25 + 25       →  50               // numbers do real math
// old C way: atoi(code.c_str())  — needs a C string
```

> ⚠ EXAM WATCH
> stoi() is not written in the notes — the conversion line is a comment for YOU to complete. One line does it: temp = stoi(code);

LAST MINUTE

60-Second Recall

TOPIC	ONE-LINE HOOK
01 Classes & objects	Blank form → filled form. Private = locked locker; public = set/get buttons.
02 Timetable	LEGO: session → course → student. Bug: course clashes with itself — skip i == k.
03 Portable	TV remote: change the inside, keep the buttons — nobody breaks.
04 The arrow	Object → dot. Pointer → arrow. ptr = &obj connects them.
05 new	Hotel room with no name — room number (pointer) only. new BEFORE use.
06 Speed	Ship the address, not the house. Same object — function can change it.
07 Strings	\0 = invisible full-stop. c_str() = bridge for old C functions like open().
08 Convert	stoi("42") → 42. to_string(42) → "42". Text glues, numbers add.

You scored 7/7 on the quick checks in our session — walk in calm. 🍀